Kỹ thuật Lập trình Thi đấu

Chủ đề: CHIA ĐỂ TRỊ (DIVIDE AND CONQUER)

Chuyên đề Thuật toán

August 17, 2026

Nội dung bài học

1. Bài toán mở đầu
2. Tiếp cận sơ cấp đến cao cấp
3. Tư tưởng Chia để trị
4. Độ phức tạp
5. Ví dụ kinh điển 1: Lũy thừa nhị phân
6. Ví dụ kinh điển 2: Merge Sort
7. So sánh & Mở rộng
8. Tổng kết và Bài tập

Đặt vấn đề

Bài toán: Tính lũy thừa (Exponentiation)

Cho hai số nguyên dương a và n. Hãy tính giá trị của $a^n \pmod{10^9 + 7}$.
Ví dụ: $a = 2, n = 13 \implies 2^{13} = 8192$.

Cách tiếp cận trực tiếp (Ngây thơ)

Nhân a với chính nó n lần:

$$a^n = \underbrace{a \times a \times \cdots \times a}_{n \text{ lần}}$$

Nút thắt (Bottleneck)

Nếu $n \approx 10^{18}$, vòng lặp sẽ chạy 10^{18} bước. Máy tính thông thường (chạy $\sim 10^8$ phép tính/giây) sẽ mất hàng chục năm để tính xong!

Cách giải trực tiếp & Quan sát

Listing 1: Cách ngây thơ $\mathcal{O}(n)$

```
long long power(long long a,
                long long n) {
    long long res = 1;
    for (long long i = 0; i < n; ++i)
        res *= a;
    return res;
}
```

Ý tưởng thuật toán

Thay vì tính a^n từ đầu, ta có thể tận dụng kết quả của bài toán nhỏ hơn không? Nhận xét:

$$a^{10} = a^5 \times a^5$$

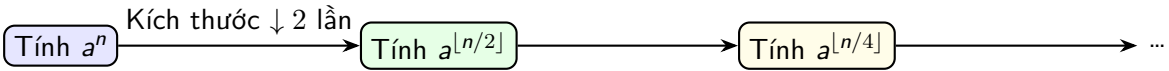
Ta chỉ cần tính a^5 **một lần**, sau đó bình phương lên!

Phân tích toán học làm giảm kích thước

Biến đổi bài toán

Tính chẵn lẻ của n cho ta tính chất sau:

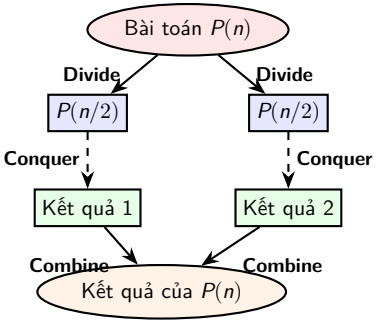
- Nếu n chẵn: $a^n = \left(a^{\frac{n}{2}}\right)^2$
- Nếu n lẻ: $a^n = a \times \left(a^{\frac{n-1}{2}}\right)^2$



Chia để trị (Divide and Conquer) là gì?

Định nghĩa tổng quát

Là phương pháp thiết kế thuật toán giải quyết bài toán lớn bằng cách chia nó thành các bài toán con nhỏ hơn (cùng loại), giải chúng, rồi kết hợp kết quả lại.



Mô hình đệ quy tổng quát

Listing 2: Pseudocode Chia để trị

```
def Solve(problem):
    # 1. Base case
    if problem is small:
        return direct_solution(problem)

    # 2. Divide
    subprobs = split(problem)

    # 3. Conquer (De quy)
    res = []
    for sub in subprobs:
        res.append(Solve(sub))

    # 4. Combine
    return combine(res)
```

Ba bước cốt lõi

1. **Divide (Chia):** Tách bài toán thành a bài toán con kích thước n/b.
2. **Conquer (Trị):** Giải đệ quy bài toán con.
3. **Combine (Kết hợp):** Trộn đáp án để tạo ra đáp án bài lớn.

Từ cây đệ quy đến định lý Master

Phương trình đệ quy (Recurrence Relation)

Thời gian chạy $T(n)$ của thuật toán Chia để trị thường có dạng:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

Trong đó:

- a: Số lượng bài toán con.
- n/b: Kích thước của mỗi bài toán con.
- $f(n)$: Chi phí để **Chia** (Divide) và **Kết hợp** (Combine).

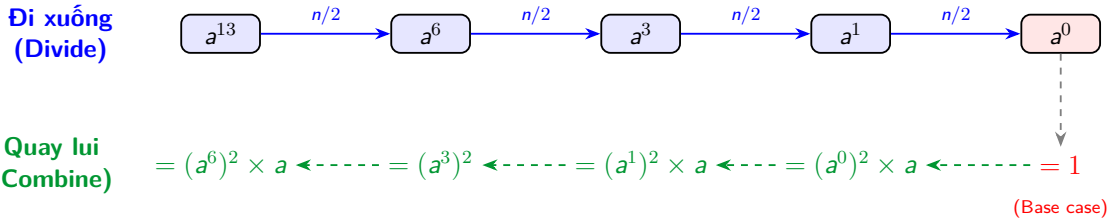
Một số trường hợp phổ biến

- $T(n) = T(n/2) + \mathcal{O}(1) \implies T(n) = \mathcal{O}(\log n)$ (Tìm kiếm nhị phân, Lũy thừa)
- $T(n) = 2T(n/2) + \mathcal{O}(n) \implies T(n) = \mathcal{O}(n \log n)$ (Merge Sort)

Mô phỏng Lũy thừa nhị phân (Binary Exponentiation)

Mô phỏng a^{13}

Quá trình đi xuống (chia kích thước) và quay lui (kết hợp).



Code & Phân tích: Lũy thừa nhị phân

Listing 3: C++ Lũy thừa nhị phân đệ quy

```cpp
long long binpow(long long a, long long n) {
    // 1. Base case
    if (n == 0) return 1;

    // 2. Divide & Conquer
    long long half = binpow(a, n / 2);

    // 3. Combine
    if (n % 2 == 0)
        return half * half;
    else
        return half * half * a;
}
```

Độ phức tạp

Mỗi lần gọi đệ quy, n giảm đi một nửa.

$$T(n) = T(n/2) + \mathcal{O}(1)$$

$$\Longrightarrow \mathcal{O}(\log n).$$

Lưu ý (Modulo)

Trong thi đấu, thường yêu cầu modulo M. Hãy '% M' ở **từng phép nhân** để tránh tràn số.

Bài toán Sắp xếp & Quan sát

Bài toán Sắp xếp

Cho mảng $A = [8, 3, 5, 1, 7, 2, 6, 4]$. Hãy sắp xếp mảng tăng dần.

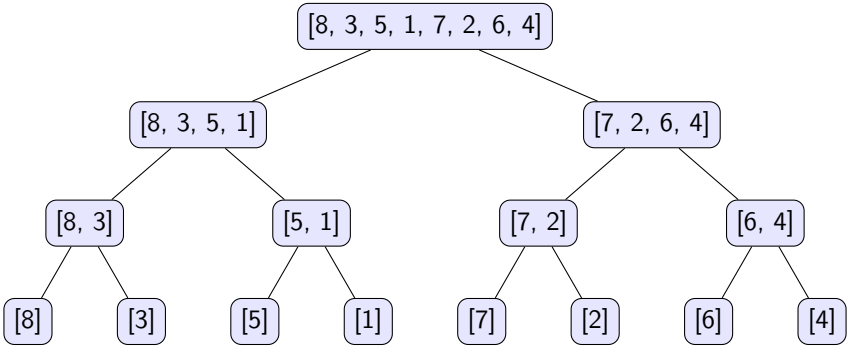
Các thuật toán sơ cấp

Bubble Sort, Selection Sort, Insertion Sort đều mất $\mathcal{O}(n^2)$. Quá chậm với $n = 10^5$.

Ý tưởng chia mảng

Một mảng lớn có thể cắt đôi. Nếu ta có thể sắp xếp riêng hai nửa, và trộn (merge) chúng lại thành một mảng hoàn chỉnh một cách nhanh chóng, bài toán sẽ được giải quyết.

Quá trình Divide (Chia mảng)



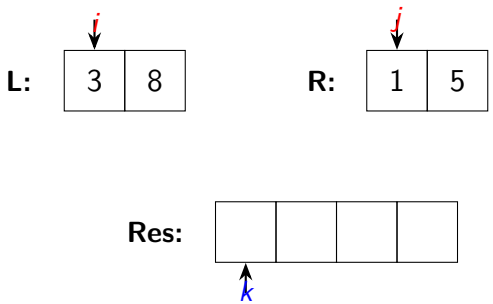
Nhận xét

Chia liên tục đến khi mảng chỉ còn 1 phần tử (hiển nhiên đã được sắp xếp).

Quá trình Combine (Trộn - Merge)

Mô phỏng gộp 2 mảng đã sắp xếp: $L = [3, 8]$ và $R = [1, 5]$

Dùng hai con trỏ i và j duyệt qua L và R, chọn phần tử nhỏ hơn đưa vào *Res*.

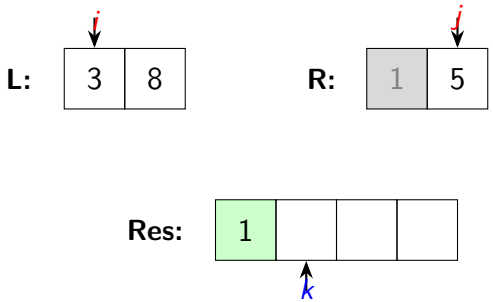


Bước 1: So sánh $L[i] = 3$ và $R[j] = 1$. Vì $1 < 3 \implies$ Chọn 1, tăng j và k.

Quá trình Combine (Trộn - Merge)

Mô phỏng gộp 2 mảng đã sắp xếp: $L = [3, 8]$ và $R = [1, 5]$

Dùng hai con trỏ i và j duyệt qua L và R, chọn phần tử nhỏ hơn đưa vào *Res*.

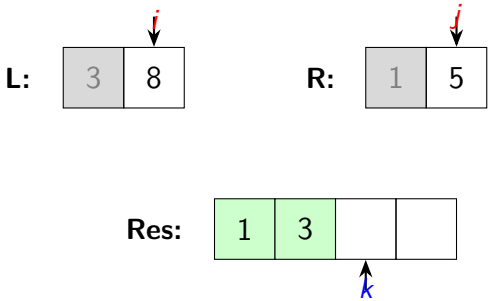


Bước 2: So sánh $L[i] = 3$ và $R[j] = 5$. Vì $3 < 5 \implies$ Chọn 3, tăng i và k.

Quá trình Combine (Trộn - Merge)

Mô phỏng gộp 2 mảng đã sắp xếp: $L = [3, 8]$ và $R = [1, 5]$

Dùng hai con trỏ i và j duyệt qua L và R, chọn phần tử nhỏ hơn đưa vào *Res*.

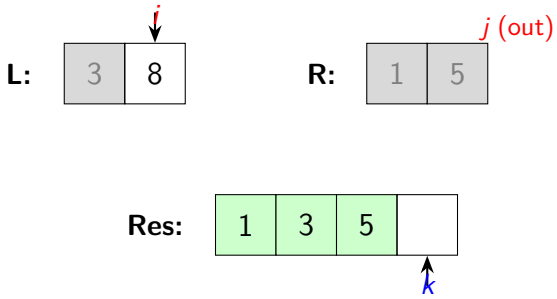


Bước 3: So sánh $L[i] = 8$ và $R[j] = 5$. Vì $5 < 8 \implies$ Chọn 5, tăng j và k.

Quá trình Combine (Trộn - Merge)

Mô phỏng gộp 2 mảng đã sắp xếp: $L = [3, 8]$ và $R = [1, 5]$

Dùng hai con trỏ i và j duyệt qua L và R, chọn phần tử nhỏ hơn đưa vào *Res*.

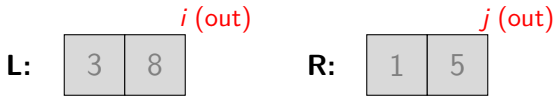


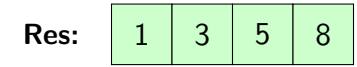
Bước 4: Mảng R đã duyệt xong. Đưa phần còn lại của L (số 8) vào *Res*.

Quá trình Combine (Trộn - Merge)

Mô phỏng gộp 2 mảng đã sắp xếp: $L = [3, 8]$ và $R = [1, 5]$

Dùng hai con trỏ *i* và *j* duyệt qua *L* và *R*, chọn phần tử nhỏ hơn đưa vào *Res*.





Kết thúc: Mảng *Res* đã được gộp và sắp xếp hoàn chỉnh!

Code C++: Cấu trúc Merge Sort

```cpp
void mergeSort(vector<int>& a, int l, int r)
     {
    // Base case: mang co <= 1 phan tu
    if (l >= r) return;

    // Divide: Tim diem chia
    int mid = l + (r - l) / 2;

    // Conquer: Goi de quy 2 nua
    mergeSort(a, l, mid);
    mergeSort(a, mid + 1, r);

    // Combine: Tron 2 nua da sap xep
    merge(a, l, mid, r);
}
```

Hàm merge

Hàm merge sẽ sử dụng mảng tạm để lưu 2 đoạn, chạy 2 con trỏ, mất chi phí $\mathcal{O}(len)$.

So sánh hai ví dụ kinh điển

c

Đặc điểm	Binary Exponentiation	Merge Sort
Cách chia	$n \to n/2$	Mảng $\to$ 2 nửa trái/phải
Số lượng bài toán con	1	2
Thao tác Combine	Phép nhân/bình phương	Dùng 2 con trỏ trộn mảng
Độ phức tạp Combine	$\mathcal{O}(1)$	$\mathcal{O}(\text{len})$
Độ phức tạp tổng	$\mathcal{O}(\log n)$	$\mathcal{O}(n \log n)$

Nhận xét quan trọng

Chia để trị **không phải** là một thuật toán duy nhất. Nó là một **chiến lược thiết kế**. Khác biệt nằm ở cách ta chia nhỏ và đặc biệt là cách ta *Combine (kết hợp)*.

Khi nào nghĩ đến Chia để trị?

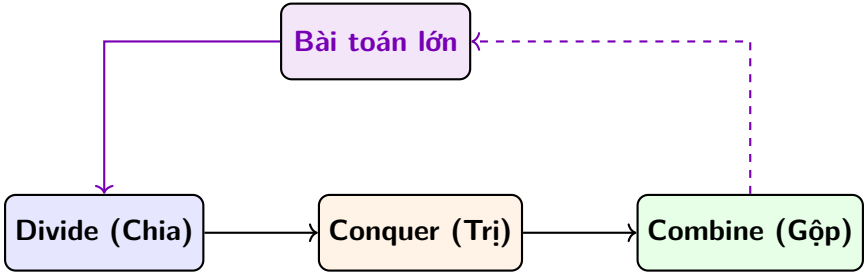
Checklist nhận diện D&C

- Bài toán lớn có thể chia thành các bài toán nhỏ hơn không?
- Các phần tử nhỏ có **cấu trúc tương tự** bài toán ban đầu không?
- Kích thước bài toán có **giảm đáng kể** (thường là chia đôi) sau mỗi lần cắt?
- Có thể **giải độc lập** các bài toán con không?
- CÓ CÁCH NÀO để **gộp kết quả** một cách hiệu quả không?

Phân biệt với Quy hoạch động (Dynamic Programming)

Trong **D&C**, các bài toán con là **độc lập** và không trùng lặp (non-overlapping).
Trong **DP**, các bài toán con **trùng lặp** (overlapping) và cần lưu mảng nhớ (memoization) để tránh tính lại.

Công thức tư duy: Chia để trị



Mở rộng

Tư tưởng Chia để trị còn là cốt lõi của rất nhiều thuật toán, cấu trúc dữ liệu mạnh mẽ: Tìm kiếm nhị phân (Binary Search), Segment Tree, Quick Sort, FFT, Đếm số nghịch thế (Inversion Count), v.v.

Q & A

Cảm ơn các bạn đã lắng nghe!